Binary Classification for Cyberattack Detection

Kevin Villon

kv56727n@pace.edu

Seidenberg School Of Computer Science and

Information Systems - Pace University (CS 619)

Instructor: Dr. Benjamin

05/14/26

Introduction

This project examines whether machine learning classifiers can distinguish between normal network traffic and malicious network traffic in an Industrial Internet of Things (IIoT) environment. IIoT systems consist of interconnected devices, sensors, and industrial control systems that communicate over networks to monitor, automate, and manage industrial operations. Examples of these systems include factory sensors, predictive maintenance systems, utility grid monitoring systems, smart manufacturing equipment, and industrial automation systems that continuously exchange data in real time to improve efficiency and operational reliability (Matterport, 2025). Because these systems constantly transmit and receive large amounts of network data, malicious activity can become difficult to detect within normal traffic patterns. As a result, cybersecurity monitoring and intrusion detection are important components of IIoT environments.

For this project, I used the WUSTL-IIoT-2021 dataset and focused on binary classification, where network traffic was classified as either normal or malicious. This report describes the dataset, the preprocessing steps, the classification algorithms used, and the results of the analysis. The dataset included traffic features such as packet counts, jitter, packet loss, and other important attributes. Before training the models, I removed unnecessary attributes, converted the *Target* attribute from numeric to nominal, and split the dataset into training and testing files. I used three Weka classifiers for data mining: J48, IBk, and DecisionTable. Overall, all three models performed very well, with each reaching above 99.98% accuracy.

Dataset Description

For my final project, the WUSTL-IIoT-2021 dataset was used and can be obtained from the following URL: https://www.cse.wustl.edu/~jain/iiot2/index.html

The dataset was compiled for cybersecurity research and intrusion detection analysis. It includes both normal network traffic and multiple categories of cyberattacks, allowing researchers to analyze differences between legitimate and malicious behavior. The dataset was originally obtained as a CSV file and contains a large number of instances and multiple attributes that describe network behavior. These attributes capture characteristics such as packet counts, byte counts, timing information, protocol types, and transmission behavior.

The prepared dataset used in this project included the attributes shown in the table below. Most of the traffic feature descriptions are based on the selected feature table provided by the WUSTL-IIoT-2021 researchers (Zolanvari et al., 2021):

Attribute	Type	Description
Mean	Numeric	Average time that active network traffic flows lasted
Sport	Numeric	Source port number
Dport	Numeric	Destination port number
SrcPkts	Numeric	Number of source packets
DstPkts	Numeric	Number of destination packets
TotPkts	Numeric	Total packet count
DstBytes	Numeric	Destination byte count
SrcBytes	Numeric	Source byte count
TotBytes	Numeric	Total bytes transferred
SrcLoad	Numeric	Source bits per second
DstLoad	Numeric	Destination bits per second
Load	Numeric	Total bits per second
SrcRate	Numeric	Source packets per second
DstRate	Numeric	Destination packets per second
Rate	Numeric	Total packets per second
SrcLoss	Numeric	Source packets retransmitted or dropped

DstLoss	Numeric	Destination packets retransmitted or dropped
Loss	Numeric	Total packets retransmitted or dropped
pLoss	Numeric	Percent packet loss
SrcJitter	Numeric	Source jitter in milliseconds
DstJitter	Numeric	Destination jitter in milliseconds
SIntPkt	Numeric	Source interpacket arrival time
DIntPkt	Numeric	Destination interpacket arrival time
Proto	Numeric	Transaction protocol
Dur	Numeric	Total record duration
TcpRtt	Numeric	TCP round-trip time
IdleTime	Numeric	Time since the last packet activity
Sum	Numeric	Total accumulated duration of aggregated records
Min	Numeric	Minimum duration of aggregated records
Max	Numeric	Maximum duration of aggregated records
sDSb	Numeric	Source different service byte value
sTtl	Numeric	Source TTL value
dTtl	Numeric	Destination TTL value
SAppBytes	Numeric	Source-to-destination application bytes
DAppBytes	Numeric	Destination-to-source application bytes
TotAppByte	Numeric	Total application bytes
SynAck	Numeric	TCP SYN-ACK connection setup time
RunTime	Numeric	Total active flow runtime
sTos	Numeric	Source type-of-service byte value
SrcJitAct	Numeric	Source active jitter value

DstJitAct	Numeric	Destination active jitter value
Target	Numeric → Nominal	Binary class label indicating normal (0) or attack (1) traffic. This attribute was converted to nominal format during preprocessing.

The original dataset also included several additional attributes related to timestamps, identifiers, and traffic labeling. Some of these attributes were removed during preprocessing because they were not needed for the binary classification task. These attributes are described below:

Attribute	**Type**	**Description**
StartTime	Nominal	Timestamp indicating when the network flow began
LastTime	Nominal	Timestamp indicating when the network flow ended
SrcAddr	Nominal	Source IP address associated with the network traffic
DstAddr	Nominal	Destination IP address associated with the network traffic
sIpId	Numeric	Source IP identifier value
dIpId	Numeric	Destination IP identifier value

Traffic	Nominal	Original traffic category label identifying specific traffic types such as normal, DoS, reconnaissance, backdoor, or command injection

In addition to these attributes, the original dataset also included the *Target* attribute as a numeric attribute, with 0 representing normal traffic and 1 representing malicious traffic. During preprocessing, *Target* was converted to nominal format so that Weka would treat it as a categorical class label for classification.

These attributes provide useful information for identifying patterns in network activity. For example, abnormal values in packet rates, unusual high or low idle times, or irregular communication patterns may indicate the presence of a cyberattack. By using these attributes, machine learning models can learn the differences between normal and attack traffic, making them valuable for detecting potential security threats in IIoT environments.

Data Preparation

Before running the classification algorithms, I used Weka to prepare the dataset for data mining. Weka is an open-source machine learning and data mining software tool developed at the University of Waikato in New Zealand. It provides tools for preprocessing data, training machine learning models, and evaluating model performance (GeeksforGeeks, 2025a). I first loaded the original CSV file into Weka and saved it as an ARFF file. ARFF is Weka's Attribute-Relation File Format, which is used to store datasets for machine learning analysis in Weka.

The original dataset contained several attributes that could cause the model to rely on identifiers instead of actual network behavior. The researchers of the dataset also included a special note to remove these attributes "as they are unique to the attacks and would expose the type of the attack to the model" (Zolanvari et al., 2021). *StartTime*, *LastTime*, *SrcAddr*, *DstAddr*, *sIpId*, and *dIpId* were excluded because they describe when the traffic occurred or identify specific source and destination information, rather than general patterns of network behavior. Removing these attributes helped reduce the chance that the models would memorize specific records instead of learning patterns that could generalize to new traffic.

I also removed the *Traffic* attribute before training the models. Although this attribute identifies the specific traffic category, such as normal, DoS, reconnaissance, backdoor, or command injection, my project focuses on binary classification using the *Target* attribute. Since *Target* already labels each instance as normal or malicious, keeping *Traffic* could have allowed the model to rely too heavily on the original traffic category instead of learning from the network features. This issue is an example of data leakage, because the model could use an answer-like

attribute instead of learning from the actual network behavior features. Data leakage happens when a model learns from information that should not be included in the training process, which can make the model's performance appear better than it actually is and reduce its effectiveness on new, unseen data (GeeksforGeeks, 2025b). Thus, *Target* was kept as the class label, while *Traffic* was removed from the final training and testing files.

The *Target* attribute was originally stored as a numeric type, with 0 representing normal traffic and 1 representing attack traffic. Since this project uses classification learning, I converted Target into a nominal class attribute in Weka. This allowed Weka to treat the values as categories and avoid treating the task as a numeric prediction problem (Frank et al., 2016, pp. 111–112). After these changes were made, the dataset contained the network behavior attributes used for prediction and the Target attribute used as the class label.

Finally, I separated the dataset into two files: one for training and one for testing. The training set contained 238,893 instances, while the testing set contained 955,571 instances. The training file was used to build the classification models, and the testing file was used to evaluate how well each model performed on data it had not already seen.

Data Analysis

For this project, I performed classification learning using Weka. According to *Data mining: Practical machine learning tools and techniques*, classification learning involves giving a learning scheme a set of classified examples so that it can learn how to classify unseen examples (Frank et al., 2016, pp. 59). In this project, the classified examples were IIoT network traffic records labeled as either normal or malicious, and the goal was to train models that could classify new traffic records based on their network behavior attributes. The class attribute used for this task was *Target*, where 0 represents normal traffic and 1 represents malicious traffic. Since this was a supervised learning problem, the models were trained using examples that already had known class labels. The trained models were then tested on a separate test file to evaluate how well they could classify new, unseen network traffic.

I used three classification algorithms in Weka: J48, IBk, and DecisionTable. I chose these algorithms because they represent different approaches to classification. J48 uses a decision-tree approach, IBk uses similarity-based instance learning, and DecisionTable uses a table/rule-based method. Using three different classifiers allowed me to compare how different model types handled the same IIoT network traffic data.

J48

The first classifier I used was J48, which is Weka's implementation of the C4.5 decision tree algorithm. In *C4.5: Programs for machine learning*, the author J. Ross Quinlan explains that a

decision tree contains decision nodes, branches, and leaves. A decision node tests an attribute, each branch represents an outcome of that test, and a leaf indicates the predicted class. A case is classified by starting at the root of the tree and following the appropriate branches until a leaf is reached (Quinlan, 1993, pp. 5–6). The C4.5 algorithm builds a decision tree by repeatedly testing attributes and splitting the training cases into smaller subsets until the model can assign a final class label.

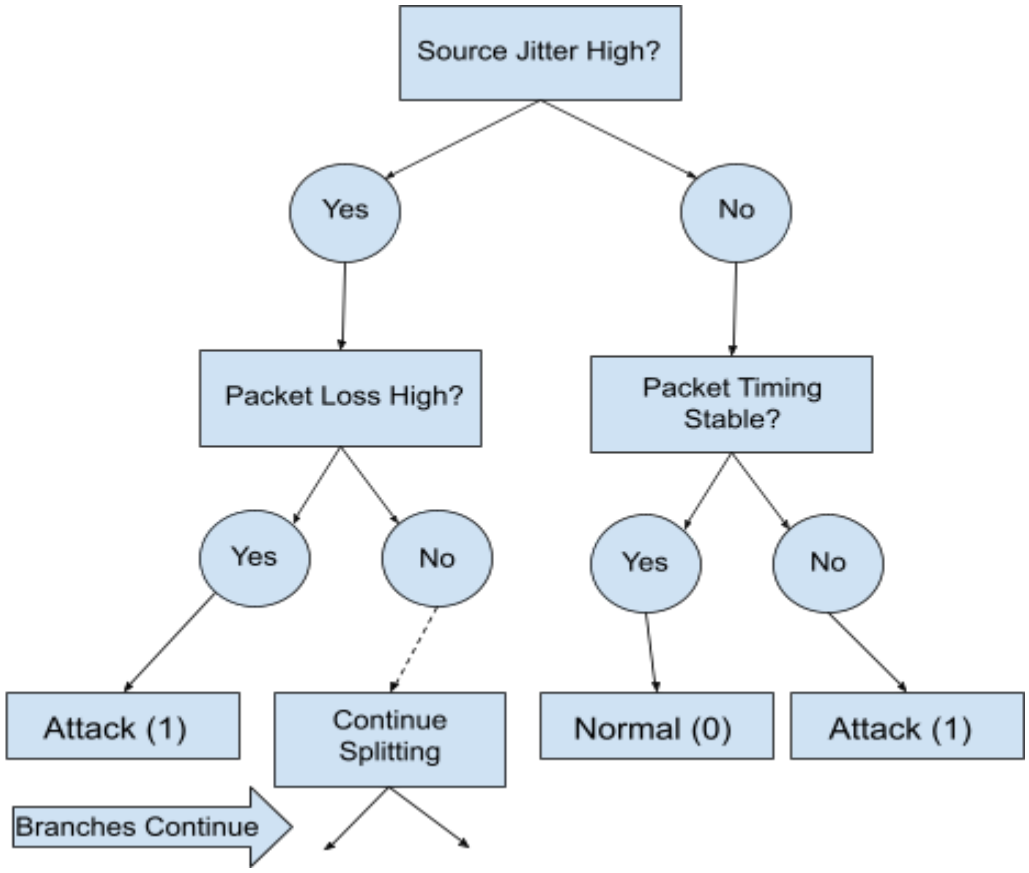


Figure 1. Example of a decision tree using jitter and packet behavior to classify network traffic

Figure 1 shows an example of how jitter could be used in a decision-tree model. This figure was created as an illustration and does not represent the full J48 model produced by Weka. Jitter refers to variation in packet timing or delay. In cybersecurity traffic analysis, timing patterns can be useful because some malicious beaconing activity uses jitter to make communication intervals less predictable and harder to detect (Garmiza, 2025). In this project, jitter-related attributes were included as network behavior features that may help the classifier distinguish between normal and malicious traffic. This makes J48 useful for this project because it can produce an interpretable model showing which traffic attributes, such as packet rates, ports, or timing behavior, were used to separate normal traffic from attack traffic.

IBk

The second classifier I used was the IBk algorithm, which is Weka's k-nearest neighbors classifier. IBk is an instance-based learning method. Unlike J48, it does not build a decision tree, and unlike DecisionTable, it does not create a table of rules. Instead, it stores the training instances and uses them as examples when classifying new data. When a new instance needs to be classified, IBk compares it to the examples in the training data and looks for the nearest neighbor or group of nearest neighbors based on similarity. According to *Instance-Based Learning Algorithms*, instance-based learning generates classification predictions using specific stored instances rather than abstract rules or trees. The authors also explain that these algorithms use similarity functions to compare instances during classification (Aha et al., 1991, pp. 37, 40–41). In this project, IBk was useful because it tested whether normal and malicious IIoT traffic records were similar to other records in the same class based on their network behavior features.

A common way to measure similarity in nearest-neighbor classification is Euclidean distance. Euclidean distance calculates how far apart two instances are based on their numeric attribute values. In an example using *SrcJitter* and *pLoss*, the distance between a new instance and a stored training example can be calculated as:

$$\textit{Euclidean Distance Formula}:$$

$$d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

For example, in Figure 3, the new instance has *SrcJitter* = 700 and *pLoss* = 30. One nearby attack example has *SrcJitter* = 900 and *pLoss* = 35. The distance would be:

$$\sqrt{(700 - 900)^2 + (30 - 35)^2}$$

$$\sqrt{(-200)^2 + (-5)^2}$$

$$\sqrt{40000 + 25} = \sqrt{40025} \approx 200.06$$

This shows that the new instance is relatively close to the attack example in the simplified two-dimensional space. Of course, in the actual IBk process, the classifier calculates the distance between the new instance and the training examples, then selects the nearest neighbor or group of nearest neighbors.

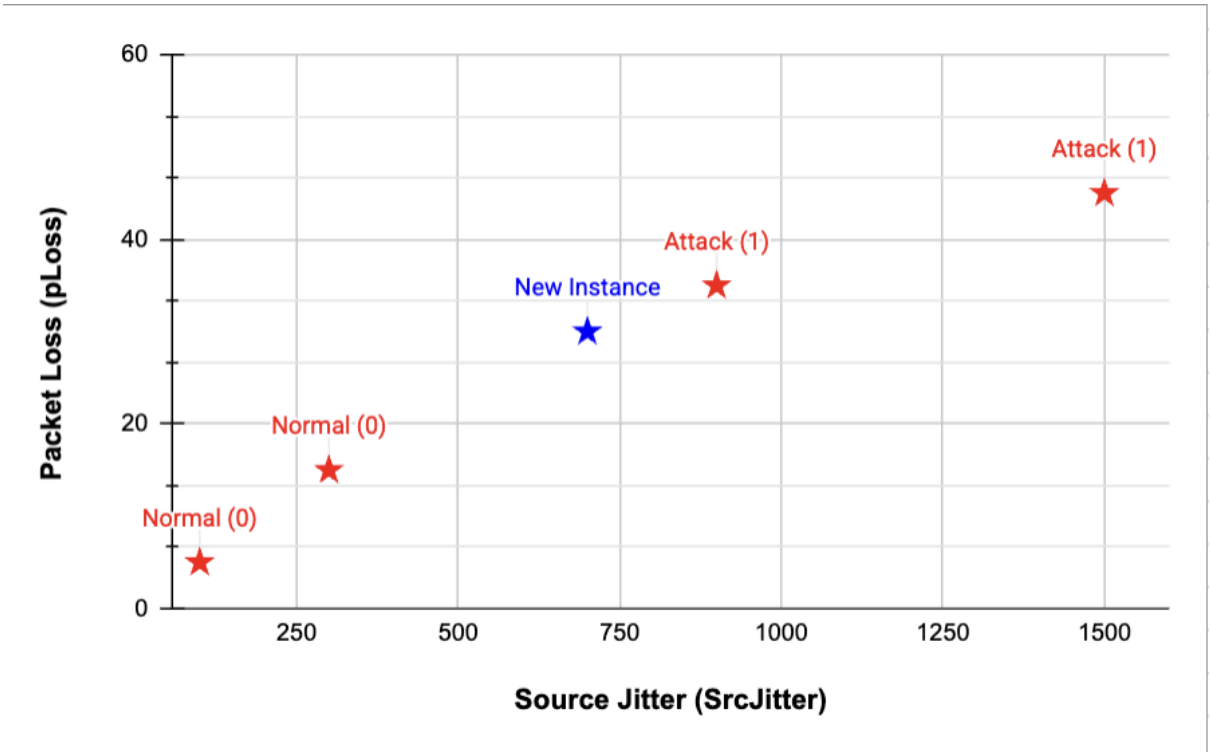


Figure 2. Simplified scatter plot example of IBk classification using source jitter and packet loss

Figure 2 shows a simplified scatter plot example of how IBk can classify a new network traffic instance using source jitter and packet loss. Each plotted red point represents a stored training example, while the blue point represents a new instance that needs to be classified. Since the new instance is closest to an attack example, IBk would likely classify it as Attack (1) in this simplified example. This figure is only an illustration and does not represent the full IBk model used by Weka for this project.

DecisionTable

The third classifier I used was the DecisionTable algorithm. This classifier builds a simple table-based model using selected attributes. Unlike IBk, DecisionTable does not rely on distance calculations. Instead, it uses selected attributes to match a new instance to table entries and predict the class based on the matching examples. In *The Power of Decision Tables*, Ron Kohavi explains that a decision table has two main parts: a schema, which is the set of features included in the table, and a body, which contains labeled instances based on those selected features. When classifying a new instance, the decision table searches for matching examples using only the selected features.If the new instance matches one or more rows in the decision table, the classifier chooses the class that appears most often among those matching rows. If there is no matching row, the classifier uses the most common class from the overall training data (Kohavi, 1995, pp. 175–176).

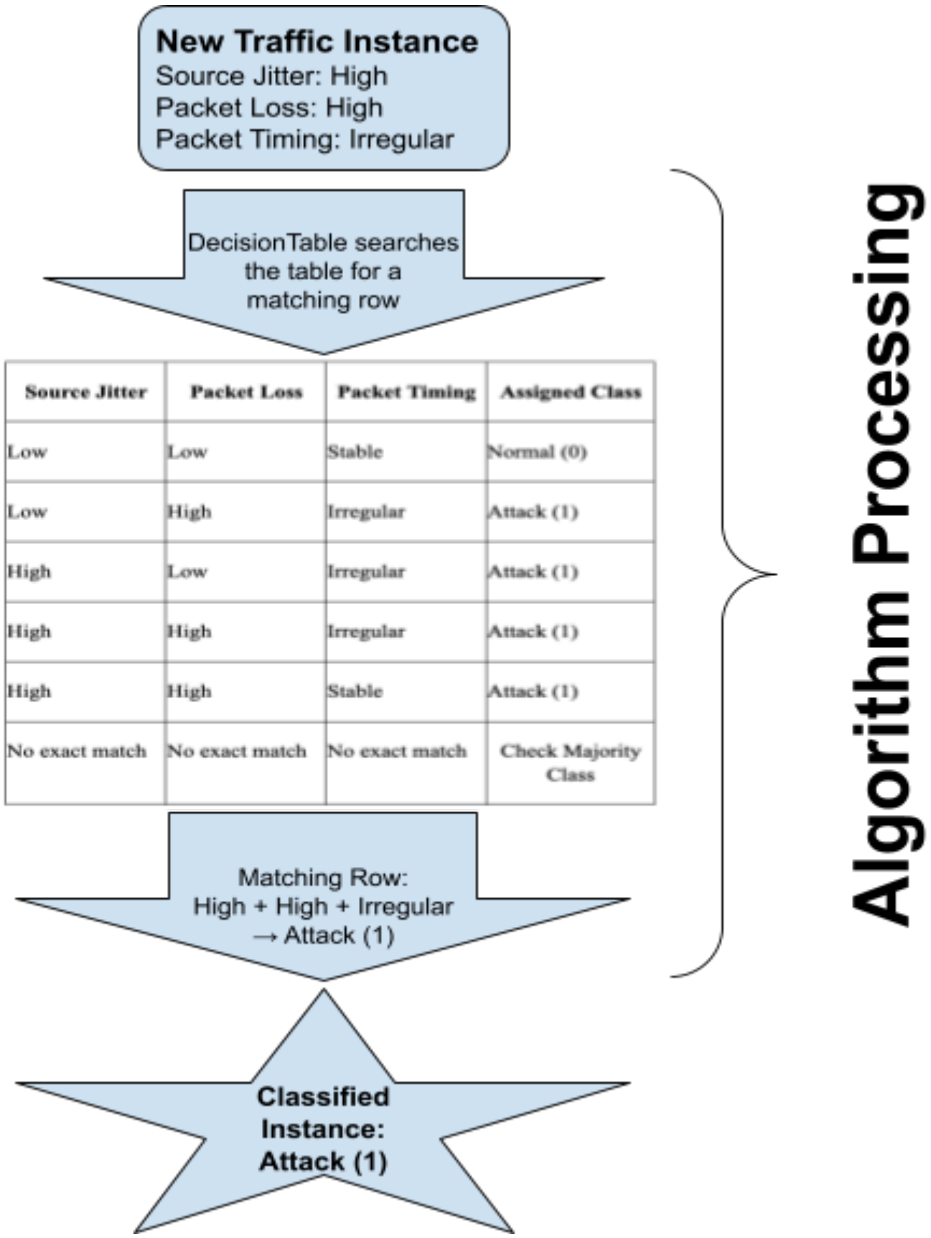


Figure 3. Flowchart of how DecisionTable classifies a new traffic instance by matching selected attribute values.

Figure 3 shows a flowchart of how the DecisionTable classifier can be used to classify network traffic. In this example, the classifier examines the selected attributes being Source Jitter, Packet Loss, and Packet Timing and searches the table for a matching row. Because the new instance matches the row with the values High, High, and Irregular, the classifier assigns the class Attack (1). The figure also shows that if no exact match is found, the classifier can fall back on the majority class. This is only an illustrative example and does not represent the full DecisionTable model produced by Weka.

Results

The three classifiers were evaluated using the prepared test file in Weka. The class label was *Target*, where 0 represented normal traffic and 1 represented attack traffic. Overall, all three models performed very well on the test examples, with each classifier achieving above 99.98% accuracy.

Classifier	Correctly Classified	Incorrectly Classified	Accuracy	Build Time	Test Time
J48	955,482	89	99.9907%	8.9 sec	3.99 sec
IBk	955,459	112	99.9883%	0.06 sec	12,025.66 sec
DecisionTable	955,460	111	99.9884%	30.4 sec	5.04 sec

J48 had the highest accuracy and the lowest number of incorrect classifications. DecisionTable was very close to IBk in accuracy but tested much faster. IBk had a very short build time, but it took much longer to test because nearest-neighbor methods compare new instances to stored training examples.

J48 Results

The J48 model produced a pruned decision tree with 21 leaves and a total tree size of 41. It was trained using the available network behavior attributes in the prepared dataset. However, the final decision tree only used some of those attributes as split points, including *sTtl*, *Sport*, *DstRate*, *Dur*, *SIntPkt*, *SrcLoad*, *pLoss*, *SrcJitAct*, *SrcPkts*, *Dport*, *Proto*, and *Mean*.

Confusion Matrix (J48)

Actual Class	Predicted Normal (0)	Predicted Attack (1)
Normal (0)	885,880	25
Attack (1)	64	69,602

J48 correctly classified most normal and attack examples. It correctly classified 955,482 out of 955,571 test instances, giving it an accuracy of 99.9907%. It misclassified 25 normal instances as attacks and 64 attack instances as normal. This means the model made slightly more false negatives than false positives. A false positive is like a "false alarm," where the model identifies something as positive even though it is not actually positive. In this project, a false positive means normal traffic was incorrectly classified as attack traffic. A false negative means the model says something is negative when it is actually positive. In this project, a false negative means attack traffic was incorrectly classified as normal traffic (University of Hawai‘i at Mānoa, n.d.). Since false negatives represent missed attacks, they are especially important in cybersecurity because malicious activity could go undetected.

The J48 model performed well on the test dataset and correctly detected most attack traffic. However, even a small number of missed attacks can matter in cybersecurity. Because of this, the model's results are favorable, but they should be viewed cautiously until the model is tested on additional IIoT traffic data.

IBk Results

The IBk model used one nearest neighbor for classification. As explained in the Data Analysis section, IBk compares a new instance to stored training examples using similarity or distance. In this run, Weka classified each test instance by using the class label of the single closest training example. The model built very quickly, taking only 0.06 seconds, but the testing time was much longer at 12,025.66 seconds (about 3 hours) because IBk compares test instances to stored training examples during classification.

Confusion Matrix (IBk)

Actual Class	**Predicted Normal (0)**	**Predicted Attack (1)**
Normal (0)	885,854	51
Attack (1)	61	69,605

IBk correctly classified 955,459 out of 955,571 test instances, giving it an accuracy of 99.9883%. It made 112 total errors. The model misclassified 51 normal instances as attacks and 61 attack instances as normal. Compared with J48, IBk had slightly more total errors, but the difference was very small.

The IBk result makes sense because many network traffic records may be similar to other records in the same class. If attack traffic examples are close to other attack examples based on their network behavior features, IBk may classify them correctly. However, the long testing time shows a limitation of IBk on large datasets. Even though it performed well, it may be less efficient than J48 or DecisionTable when many test examples must be classified. In cybersecurity, faster classification can be important because results may need to be available quickly for investigation or response.

DecisionTable Results

The DecisionTable model produced 49 rules. Weka reported that the selected feature set included attributes such as *Dport*, *Proto*, and *sTtl* along with the class label. This suggests that the model was able to classify traffic using a smaller set of selected attributes rather than relying on every attribute in the dataset.

Confusion Matrix (DecisionTable)

Actual Class	**Predicted Normal (0)**	**Predicted Attack (1)**
Normal (0)	885,905	0
Attack (1)	111	69,555

DecisionTable correctly classified 955,460 out of 955,571 test instances, giving it an accuracy of 99.9884%. It correctly classified all normal instances in the test set, with 0 normal instances misclassified as attacks. However, it misclassified 111 attack instances as normal. This means DecisionTable had no false positives in this test, but it had more false negatives than J48 and IBk.

These results suggest that DecisionTable was less likely to label traffic as an attack compared with the other classifiers. Although the model is simpler and easier to interpret, it also resulted in more attack instances being classified as normal compared with J48 and IBk. This is important because, in cybersecurity, missing an attack can be more serious than incorrectly flagging normal traffic.

Conclusion

This project showed that machine learning classifiers can be useful for distinguishing between normal and malicious IIoT network traffic when trained on traffic behavior attributes. The WUSTL-IIoT-2021 dataset provided network features that helped the models learn patterns related to normal and attack traffic. All three classifiers performed very well on the test dataset, but J48 had the strongest overall performance because it had the highest accuracy, the fewest total errors, reasonable testing time, and an interpretable decision-tree structure. IBk performed almost as well as J48, but its long testing time made it less practical for a large dataset. DecisionTable was useful as a simpler comparison model, but it had the highest number of attack instances incorrectly classified as normal, which is important because false negatives represent missed attacks. Overall, the results suggest that classification learning can help identify malicious IIoT traffic. These findings should be carefully interpreted because the models were tested using data from the same dataset source. Further testing with additional IIoT traffic data would be needed to better understand how well these models would perform in other real-world environments.

References:

Aha, D. W., Kibler, D., & Albert, M. K. (1991). Instance-based learning algorithms. *Machine Learning, 6*, 37–66.

Frank, E., Hall, M. A., & Witten, I. H. (2016). *The WEKA workbench: Online appendix for "Data mining: Practical machine learning tools and techniques"* (4th ed.). Morgan Kaufmann.

Garmiza, M. (2025, June 18). *The jitter-trap: How randomness betrays the evasive*. Varonis. https://www.varonis.com/blog/jitter-trap

GeeksforGeeks. (2025a, October 28). *Introduction to Weka: Key features and applications*. https://www.geeksforgeeks.org/machine-learning/introduction-to-weka-key-features-and-applications/

GeeksforGeeks. (2025b, July 23). *What is data leakage in machine learning?* https://www.geeksforgeeks.org/machine-learning/what-is-data-leakage/

Kohavi, R. (1995). The power of decision tables. In *Machine Learning: ECML-95* (pp. 174–189). Springer

Matterport. (2025, May 22). *IIoT applications*. Matterport. https://matterport.com/blog/iiot-applications

Quinlan, J. R. (1993). *C4.5: Programs for machine learning*. Morgan Kaufmann.

University of Hawai‘i at Mānoa. (n.d.). *Practices of science: False positives and false negatives*. Exploring Our Fluid Earth. https://manoa.hawaii.edu/exploringourfluidearth/chemical/matter/properties-matter/practices-science-false-positives-and-false-negatives

Zolanvari, M., Teixeira, M. A., Gupta, L., Khan, K. M., & Jain, R. (2021, October). *WUSTL-IIOT-2021 dataset for IIoT cybersecurity research*. Washington University in St. Louis. http://www.cse.wustl.edu/~jain/iiot2/index.html

Appendix A: J48 Model Output

```
=== Run information ===

Scheme:       weka.classifiers.trees.J48 -C 0.25 -M 2
Relation:
wustl_iiot_2021-weka.filters.unsupervised.attribute.Remove-R1-4,38-39-weka.fil
ters.unsupervised.instance.RemovePercentage-P20.0-V-weka.filters.unsupervised.
attribute.NumericToNominal-Rlast-weka.filters.unsupervised.attribute.Remove-R4
2
Instances:    238893
Attributes:   42
              Mean
              Sport
              Dport
              SrcPkts
              DstPkts
              TotPkts
              DstBytes
              SrcBytes
              TotBytes
              SrcLoad
              DstLoad
              Load
              SrcRate
              DstRate
              Rate
              SrcLoss
              DstLoss
              Loss
              pLoss
              SrcJitter
              DstJitter
              SIntPkt
              DIntPkt
              Proto
              Dur
              TcpRtt
              IdleTime
              Sum
              Min
              Max
```

```
               sDSb
               sTtl
               dTtl
               SAppBytes
               DAppBytes
               TotAppByte
               SynAck
               RunTime
               sTos
               SrcJitAct
               DstJitAct
               Target
Test mode:    user supplied test set:  size unknown (reading incrementally)

=== Classifier model (full training set) ===

J48 pruned tree
------------------

sTtl <= 128
|   sTtl <= 127
|   |   Sport <= 9299
|   |   |   DstRate <= 4901.960938: 0 (3295.0/5.0)
|   |   |   DstRate > 4901.960938
|   |   |   |   Dur <= 0.000032: 1 (10.0)
|   |   |   |   Dur > 0.000032
|   |   |   |   |   SIntPkt <= 0.005: 0 (9.0)
|   |   |   |   |   SIntPkt > 0.005
|   |   |   |   |   |   SrcLoad <= 2480000: 1 (2.0)
|   |   |   |   |   |   SrcLoad > 2480000
|   |   |   |   |   |   |   Dur <= 0.000158
|   |   |   |   |   |   |   |   SIntPkt <= 0.009: 1 (2.0)
|   |   |   |   |   |   |   |   SIntPkt > 0.009: 0 (2.0)
|   |   |   |   |   |   |   Dur > 0.000158: 0 (7.0)
|   |   Sport > 9299
|   |   |   sTtl <= 1: 0 (63.0)
|   |   |   sTtl > 1: 1 (3507.0)
|   sTtl > 127
|   |   pLoss <= 42.105263
|   |   |   SrcJitAct <= 70.405969
|   |   |   |   Dport <= 215
|   |   |   |   |   Dport <= 72: 0 (65.0)
```

```
|   |   |   |   |   Dport > 72: 1 (6.0)
|   |   |   |   Dport > 215: 0 (217971.0/2.0)
|   |   |   SrcJitAct > 70.405969
|   |   |   |   SrcPkts <= 71: 0 (128.0/1.0)
|   |   |   |   SrcPkts > 71: 1 (14.0)
|   |   pLoss > 42.105263
|   |   |   SrcPkts <= 4: 1 (103.0/2.0)
|   |   |   SrcPkts > 4
|   |   |   |   DstPkts <= 0: 1 (5.0)
|   |   |   |   DstPkts > 0: 0 (3.0)
sTtl > 128
|   sTtl <= 254: 1 (13585.0)
|   sTtl > 254
|   |   Proto <= 6: 1 (99.0)
|   |   Proto > 6
|   |   |   Mean <= 2: 0 (6.0)
|   |   |   Mean > 2: 1 (11.0)

Number of Leaves  :     21

Size of the tree :      41


Time taken to build model: 8.9 seconds

=== Evaluation on test set ===

Time taken to test model on supplied test set: 3.99 seconds

=== Summary ===

Correctly Classified Instances      955482               99.9907 %
Incorrectly Classified Instances        89                0.0093 %
Kappa statistic                          0.9993
Mean absolute error                      0.0001
Root mean squared error                  0.0096
Relative absolute error                  0.0987 %
Root relative squared error              3.6975 %
Total Number of Instances           955571

=== Detailed Accuracy By Class ===
```

```
                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC
Area  PRC Area  Class
                 1.000    0.001    1.000      1.000    1.000      0.999
1.000     1.000     0
                 0.999    0.000    1.000      0.999    0.999      0.999
1.000     0.999     1
Weighted Avg.    1.000    0.001    1.000      1.000    1.000      0.999
1.000     1.000

=== Confusion Matrix ===

      a      b   <-- classified as
 885880     25 |      a = 0
     64  69602 |      b = 1
```

Appendix B: IBk Model Output

```
=== Run information ===

Scheme:       weka.classifiers.lazy.IBk -K 1 -W 0 -A
"weka.core.neighboursearch.LinearNNSearch -A \"weka.core.EuclideanDistance -R
first-last\""
Relation:
wustl_iiot_2021-weka.filters.unsupervised.attribute.Remove-R1-4,38-39-weka.fil
ters.unsupervised.instance.RemovePercentage-P20.0-V-weka.filters.unsupervised.
attribute.NumericToNominal-Rlast-weka.filters.unsupervised.attribute.Remove-R4
2
Instances:    238893
Attributes:   42
              Mean
              Sport
              Dport
              SrcPkts
              DstPkts
              TotPkts
              DstBytes
              SrcBytes
              TotBytes
              SrcLoad
              DstLoad
              Load
              SrcRate
              DstRate
              Rate
              SrcLoss
              DstLoss
              Loss
              pLoss
              SrcJitter
              DstJitter
              SIntPkt
              DIntPkt
              Proto
              Dur
              TcpRtt
              IdleTime
              Sum
```

```
               Min
               Max
               sDSb
               sTtl
               dTtl
               SAppBytes
               DAppBytes
               TotAppByte
               SynAck
               RunTime
               sTos
               SrcJitAct
               DstJitAct
               Target
Test mode:    user supplied test set:  size unknown (reading incrementally)

=== Classifier model (full training set) ===

IB1 instance-based classifier
using 1 nearest neighbour(s) for classification


Time taken to build model: 0.06 seconds

=== Evaluation on test set ===

Time taken to test model on supplied test set: 12025.66 seconds

=== Summary ===

Correctly Classified Instances      955459               99.9883 %
Incorrectly Classified Instances       112                0.0117 %
Kappa statistic                          0.9991
Mean absolute error                      0.0001
Root mean squared error                  0.0108
Relative absolute error                  0.09   %
Root relative squared error              4.1642 %
Total Number of Instances           955571

=== Detailed Accuracy By Class ===
```

```
                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC
Area  PRC Area  Class
                 1.000    0.001    1.000      1.000    1.000      0.999
1.000     1.000     0
                 0.999    0.000    0.999      0.999    0.999      0.999
1.000     0.998     1
Weighted Avg.    1.000    0.001    1.000      1.000    1.000      0.999
1.000     1.000

=== Confusion Matrix ===

      a      b   <-- classified as
 885854     51 |      a = 0
     61  69605 |      b = 1
```

Appendix C: DecisionTable Model Output

```
=== Run information ===

Scheme:       weka.classifiers.rules.DecisionTable -X 1 -S
"weka.attributeSelection.BestFirst -D 1 -N 5"
Relation:
wustl_iiot_2021-weka.filters.unsupervised.attribute.Remove-R1-4,38-39-weka.fil
ters.unsupervised.instance.RemovePercentage-P20.0-V-weka.filters.unsupervised.
attribute.NumericToNominal-Rlast-weka.filters.unsupervised.attribute.Remove-R4
2
Instances:    238893
Attributes:   42
              Mean
              Sport
              Dport
              SrcPkts
              DstPkts
              TotPkts
              DstBytes
              SrcBytes
              TotBytes
              SrcLoad
              DstLoad
              Load
              SrcRate
              DstRate
              Rate
              SrcLoss
              DstLoss
              Loss
              pLoss
              SrcJitter
              DstJitter
              SIntPkt
              DIntPkt
              Proto
              Dur
              TcpRtt
              IdleTime
              Sum
              Min
```

```
               Max
               sDSb
               sTtl
               dTtl
               SAppBytes
               DAppBytes
               TotAppByte
               SynAck
               RunTime
               sTos
               SrcJitAct
               DstJitAct
               Target
Test mode:    user supplied test set:  size unknown (reading incrementally)

=== Classifier model (full training set) ===

Decision Table:

Number of training instances: 238893
Number of Rules : 49
Non matches covered by Majority class.
      Best first.
      Start set: no attributes
      Search direction: forward
      Stale search after 5 node expansions
      Total number of subsets evaluated: 305
      Merit of best subset found:   99.99
Evaluation (for feature selection): CV (leave one out)
Feature set: 3,24,32,42

Time taken to build model: 30.4 seconds

=== Evaluation on test set ===

Time taken to test model on supplied test set: 5.04 seconds

=== Summary ===

Correctly Classified Instances      955460               99.9884 %
Incorrectly Classified Instances       111                0.0116 %
Kappa statistic                          0.9991
```

```
Mean absolute error                       0.0004
Root mean squared error                   0.0103
Relative absolute error                   0.2616 %
Root relative squared error               3.9587 %
Total Number of Instances           955571

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC
Area  PRC Area  Class
                 1.000    0.002    1.000      1.000    1.000      0.999
1.000     1.000     0
                 0.998    0.000    1.000      0.998    0.999      0.999
1.000     1.000     1
Weighted Avg.    1.000    0.001    1.000      1.000    1.000      0.999
1.000     1.000

=== Confusion Matrix ===

      a      b   <-- classified as
 885905      0 |      a = 0
    111  69555 |      b = 1
```